Expert Systems With Applications

Scalable Edge-Centric Subgraph Learning via Pool Walks for Link Prediction

--Manuscript Draft--

Manuscript Number:	ESWA-D-25-33231
Article Type:	Full length article
Section/Category:	1.1 AI / ML model and method
Keywords:	Representation Learning; Graph neural networks; Link Prediction; Subgraph-Based Learning; Scalability
Corresponding Author:	Manizheh Ranjbar RMIT University AUSTRALIA
First Author:	Manizheh Ranjbar
Order of Authors:	Manizheh Ranjbar
	Parham Moradi
	Xiaodong Li
	Mahdi Jalili
Abstract:	Graph Neural Networks (GNNs) have achieved notable success in link prediction tasks, yet many existing methods still struggle to model pairwise relationships effectively. Subgraph-based techniques provide stronger relational modeling, but they often face scalability constraints that limit them to small neighborhoods and prevent the incorporation of broader structural dependencies. To address this scalability challenge, this paper proposes Pool Walk Subgraph-Based Learning (PWLP) that combines global and local structural information. Our method leverages a pool-walk strategy, incorporating flexible node hops and prioritizing influential nodes within subgraphs, thus capturing both proximal and distant relationships. This architecture effectively integrates structural and node-based features, while mitigating the computational overhead of converting subgraphs to line graphs by restricting subgraph size through informed candidate node selection, providing scalability across diverse graph settings. Comprehensive evaluations on standard benchmark datasets show that PWLP outperforms existing state-of-the-art approaches in key metrics.

Dear Professor Ling Wang,

Editor-in-Chief, Expert Systems with Applications

I am pleased to submit our manuscript entitled " **Scalable Edge-Centric Subgraph Learning via Pool Walks for Link Prediction**" for consideration for publication in *Expert Systems with Applications*. This work introduces PWLP, an influence-aware subgraph learning framework that combines pool-walk exploration with lightweight edge-centric encoding to improve relational modelling in link prediction tasks. PWLP adaptively identifies structurally informative nodes through walk-driven neighbourhood expansion, constructs compact induced subgraphs, and reformulates link prediction as a node classification task on induced line-graphs. This approach enables the model to capture both local and broader structural dependencies while maintaining computational efficiency. PWLP addresses key limitations of deterministic subgraph extraction and heavy line-graph transformations, providing a flexible, expressive, and efficient framework for link prediction across a range of graph types.

We have evaluated the proposed approach on a diverse collection of publicly available real-world datasets, and our results demonstrate its superior predictive performance compared to current state-of-the-art graph neural network methods. We believe that the insights and contributions presented in this work will be of significant interest to the journal's readership, especially those engaged in graph representation learning, subgraph-based link prediction, and advanced GNN methodologies.

Original Article Statement:

- This manuscript represents the authors' original research and has neither been published previously nor submitted to any other journal.
- All listed authors have thoroughly reviewed the manuscript and unanimously approved its submission to *Expert Systems with Applications*.
- We appreciate your time and consideration in reviewing our work. We are eager to receive your feedback and sincerely hope our submission aligns with the rigorous standards and scholarly expectations of your esteemed journal.

Sincerely,

Manizheh Ranjbar

RMIT University, Melbourne, Australia

On behalf of all co-authors

Scalable Edge-Centric Subgraph Learning via Pool Walks for Link Prediction

Manizheh Ranjbar[a,*], Parham Moradi[a], Xiaodong Li[b] and Mahdi Jalili[a]

[a]School of Engineering, RMIT University, Melbourne, VIC, Australia
[b]School of Computing Technologies, RMIT University, Melbourne, VIC, Australia

ARTICLE INFO

Keywords:
Representation Learning
Graph Neural Networks
Link Prediction
Subgraph-Based Learning
Scalability

ABSTRACT

Graph Neural Networks (GNNs) have achieved notable success in link prediction tasks, yet many existing methods still struggle to model pairwise relationships effectively. Subgraph-based techniques provide stronger relational modeling, but they often face scalability constraints that limit them to small neighborhoods and prevent the incorporation of broader structural dependencies. To address this scalability challenge, this paper proposes Pool Walk Subgraph-Based Learning (PWLP) that combines global and local structural information. Our method leverages a pool-walk strategy, incorporating flexible node hops and prioritizing influential nodes within subgraphs, thus capturing both proximal and distant relationships. This architecture effectively integrates structural and node-based features, while mitigating the computational overhead of converting subgraphs to line graphs by restricting subgraph size through informed candidate node selection, providing scalability across diverse graph settings. Comprehensive evaluations on standard benchmark datasets show that PWLP outperforms existing state-of-the-art approaches in key metrics.

1. Introduction

A wide range of real-world applications can be naturally modeled using graphs, such as social networks, biological interactions, and chemistry Cai et al. (2021). These graphs encapsulate important relationships between entities, which is essential to understanding complex interconnections. Link prediction (LP) aims to uncover missing or potential connections in a graph, with diverse applications including friend recommendation, gene–disease association discovery, fraud detection, and disease interaction prediction Zhu et al. (2023). Its broad applicability highlights its versatility and importance across diverse domains. Graph Neural Networks (GNNs) have shown notable effectiveness in link prediction, with early frameworks such as the Graph Autoencoder (GAE) Kipf and Welling (2016) playing a foundational role. GAE uses a Message Passing Neural Network (MPNN) to independently compute node embeddings for each node in a pair. However, this approach does not explicitly model pairwise relationships, leading to identical embeddings for nodes with similar neighborhoods, which may overlook subtle differences in their structural roles or relationships within the graph. Figure 1a highlights this limitation. The inability to differentiate between nodes with identical neighborhoods when their structural relationships differ is a critical shortcoming, especially for complex graphs where subtle variations in structure are meaningful. This limitation has driven the development of advanced methods that integrate structural features (SF) with MPNNs, enhancing GNNs by incorporating additional context about nodes' roles and positions within the graph.

Models combining SF and MPNN fall into three categories: SF-then-MPNN, SF-and-MPNN, and MPNN-then-SF Wang et al. (2024).

SF-then-MPNN approaches, such as SEAL Zhang and Chen (2018) and mLink Cai and Ji (2020), enhance link prediction by incorporating SF from the h-hop enclosing subgraph around the target node pair before applying a MPNN. While these methods effectively capture complex relationships, they often lose fine-grained details due to pooling layers. LGLP Cai et al. (2021) addresses this by removing the pooling layer and using line graph transformations to preserve structure. However, this comes with a significant computational cost, as converting subgraphs to larger line graphs increases the graph's size and complexity. Although recent methods attempt to reduce the overhead of subgraph-based models using sampling Louis et al. (2022); Yin et al. (2022), simplification techniques Louis et al. (2023), they typically operate on only 1- or 2-hop neighborhoods, sacrifice global structural context, and still risk information loss.

SF-and-MPNN methods, like Neo-GNN Yun et al. (2021) and BUDDY Chamberlain et al. (2022), decouple structural features from the MPNN. By doing so, they leverage pairwise structural features alongside node representations obtained from a single pass of MPNN. While decoupling SF and MPNN improves scalability, it often comes at the expense of expressivity, as these models may capture only simplistic structural patterns, such as merely counting common neighbors, rather than modeling richer structural features, including the common-neighbor substructures explicitly captured by more expressive methods like SEAL.

MPNN-then-SF, exemplified by the NCNC model Wang et al. (2024), first applies an MPNN to the original graph

*Corresponding author
s3985835@student.rmit.edu.au (M. Ranjbar); parham.moradi@rmit.edu.au (P. Moradi); xiaodong.li@rmit.edu.au (X. Li); mahdi.jalili@rmit.edu.au (M. Jalili)
ORCID(s): 0000-0002-9455-1959 (M. Ranjbar)

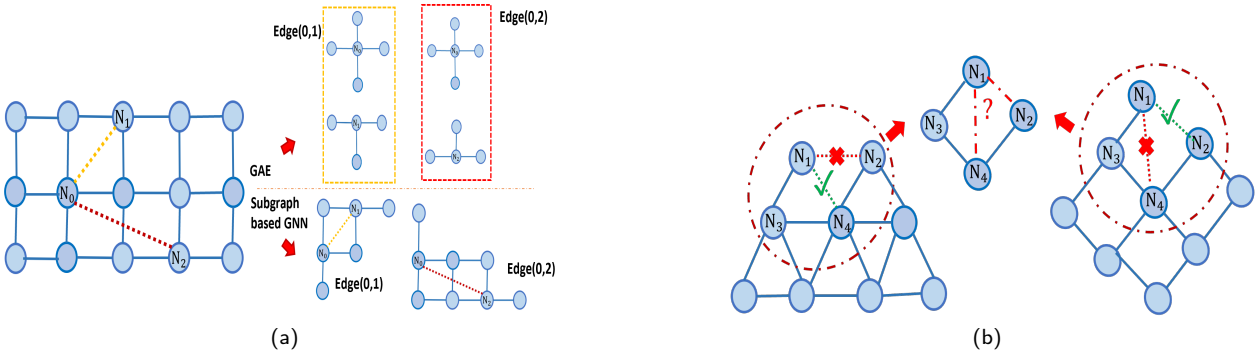


Figure 1: (a) MPNN produces identical embeddings for symmetric nodes N_1 and N_2, failing to distinguish structurally different node pairs (N_0, N_1) and (N_0, N_2). (b) Topological organizing rules are not consistent across all graphs, highlighting the importance of utilizing global structural information.

and then uses structural features to guide the pooling of node representations. This approach combines the expressiveness of SF-then-MPNN models with the scalability of SF-and-MPNN models. However, MPNN-based methods typically rely on node features for effective performance. As Hu et al. (2020) note, many real-world graphs, such as social and citation networks, contain node features that are missing or unreliable Hu et al. (2020), which poses a significant challenge for GNN-based LP models.

While GNNs can capture complex relationships when node features are informative, traditional methods remain competitive, particularly when topology serves as the primary source of information. Traditional heuristic methods, such as CN Newman (2001), AA Adamic and Adar (2003), and Katz Katz (1953), excel by utilizing structural features (e.g., shared neighbors and paths), often achieving strong performance Zhang and Chen (2018); Hu et al. (2020). However, higher-order methods that also incorporate global graph structure can further enhance performance. This highlights a key insight: link prediction is inherently a structural problem where both local and global graph information play crucial roles. Building on this understanding, subgraph-based methods have emerged as a powerful framework for modeling relational structures. To manage graph size and computational cost, subgraph-based methods often restrict the value of h to 1 or 2, focusing on capturing localized structural patterns. However, this restriction limits their ability to incorporate global information, resulting in methods that entirely overlook broader structural contexts. Figure 1b illustrates the importance of leveraging global structure due to the presence of inconsistent topological patterns across graphs. Moreover, the LGLP paper highlights an important observation: while existing methods like SEAL and LGLP can incorporate node features by concatenating them with structural features during the learning process, adding node features can sometimes degrade performance Cai et al. (2021). This further underscores the need for models that can leverage structural information effectively, with or without node attributes.

To address these challenges, we propose a novel framework that goes beyond fixed h-hop enclosing subgraphs by incorporating both global and local structural information. Our approach uses flexible walk-based pooling to prioritize influential nodes, enabling expressive and scalable edge representations capable of integrating node features when available. By transforming subgraphs into line graphs, we shift the focus from nodes to edges, enhancing the model's ability to distinguish fine-grained structural patterns and capture long-range dependencies. Extensive experiments demonstrate the effectiveness of this design across diverse graph types. Our contributions are summarized as follows:

- We propose a novel framework for generating expressive edge representations in link prediction by leveraging flexible node hops within subgraph structures. This approach captures rich contextual information and remains robust with or without node features.
- To improve expressivity and scalability, we introduce a walk pooling mechanism that prioritizes influential nodes in subgraphs, effectively managing the complexity introduced by Line Graph transformations. This design supports both dense and sparse graph settings.
- By transforming subgraphs into line graphs, we shift the learning focus from nodes to edges, improving the ability to distinguish fine-grained structural patterns.
- We validate our approach through extensive experiments, demonstrating strong predictive performance, the ability to capture neighborhood similarity and long-range dependencies (e.g., community membership), and consistent generalization across diverse graph types.

2. Related work

Link prediction approaches can be broadly classified into proximity-based (non-learning) and learning-based methods. Proximity-based methods rely on graph structure heuristics, such as Common Neighbors (CN) Newman (2001), Adamic Adar (AA) Adamic and Adar (2003), Resource Allocation (RA) Zhou et al. (2009), Significant Influence (SI) Yang et al. (2018), Shortest Path, and Katz Katz

(1953), to assign scores based on the statistical properties of nodes or paths. While these methods are simple and interpretable, they heavily rely on specific assumptions about node connectivity, which often limits their performance in complex networks. Moreover, they overlook node and edge features, further constraining their effectiveness in diverse scenarios. Learning-based methods map nodes, edges, or entire graphs into low-dimensional vector spaces, facilitating various downstream tasks. One category of these methods is embedding-based approaches, such as Matrix Factorization (MF) Menon and Elkan (2011), LINE Tang et al. (2015), DeepWalk Perozzi et al. (2014), and node2vec Qiu et al. (2018), which focus on transforming graph structures into meaningful representations. DeepWalk uses random walks combined with the Skip-gram model to generate node embeddings, while node2vec extends this by introducing biased random walks to better explore neighborhoods. LINE captures both first-order and second-order proximities, enhancing embedding quality. Despite their effectiveness in certain applications, these methods are limited by their exclusive reliance on graph structure and their transductive nature, which prevents them from generalizing to unseen nodes or graphs. This highlights the need for more advanced methods, such as those built on Graph Neural Networks (GNNs), which can seamlessly combine graph structure with node features, thereby improving predictive performance and generalizability. Graph neural networks (GNNs) have demonstrated significant potential in link prediction tasks by effectively leveraging both graph structure and node features. Methods such as Graph Convolutional Networks (GCN) Yao et al. (2019), GAT Velickovic et al. (2017), GraphSAGE Hamilton et al. (2017), and GAE Kipf and Welling (2016) focus on learning node embeddings by aggregating information from neighboring nodes via message-passing mechanisms. While these approaches capture local structural information, they often struggle to model the pairwise relationships between nodes, which are critical for accurate link prediction. To address this limitation, recent research has shifted towards methods that combine structural features and message-passing mechanisms. Key methods in this category include SEAL Zhang and Chen (2018), mLink Cai and Ji (2020), DE-GNN Li et al. (2020), and NBFNet Zhu et al. (2021), LGLP Cai et al. (2021), Neo-GNN Yun et al. (2021), BUDDY Chamberlain et al. (2022), PEG Wang et al. (2022), LGCL Zhang et al. (2023), and NCNC Wang et al. (2024). These approaches incorporate extra details, such as subgraph features, common neighbor information, and positional data, to better capture the relationships between nodes in link prediction.

3. Preliminaries

This section outlines the theoretical foundations of our study, including the Link Prediction problem, h-hop enclosing subgraphs, line graphs, Principal Component Analysis (PCA), and Graph Neural Networks (GNNs). Let $G = (V, E, A, X)$ be an undirected graph, where $V = \{1, 2, \dots, N\}$ is the set of N nodes, $E \subseteq V \times V$ is the set of existing edges, and $X \in \mathbb{R}^{N \times D}$ represents the node features. The symmetric adjacency matrix $A \in \mathbb{R}^{N \times N}$ encodes edges such that $A_{uv} = 1$ if nodes u and v are connected, and 0 otherwise.

h-hop Enclosing Subgraph. Given a pair of nodes (u, v), the h-hop enclosing subgraph is the subgraph that consists of all nodes and edges reachable within h hops from either u or v. Formally, let $N_h(u)$ and $N_h(v)$ denote the sets of nodes reachable from u and v, respectively, within h hops. The h-hop enclosing subgraph is defined as the induced subgraph $G_h(u, v) = G[N_h(u) \cup N_h(v)]$.

Line Graph. For an undirected graph $G = (V, E)$, the line graph $L(G) = (V_L, E_L)$ is formed by creating a vertex in $L(G)$ for each edge in G, meaning $V_L = E$. An edge exists between two vertices in $L(G)$ if and only if their corresponding edges in G share a common node. Thus, the edge set E_L is given by $E_L = \{(e_1, e_2) \in V_L \times V_L \mid e_1 = (u, w), e_2 = (w, v), \text{ for some } u, v, w \in V\}$.

Link Prediction Problem. Link prediction refers to the task of predicting whether an edge exists between a pair of nodes u and v in a graph G. The aim is to estimate the likelihood of a connection between these two nodes. Traditional approaches approximate the probability $p(u, v)$ using structural heuristics, such as counting the number of common neighbors. Formally, this is expressed as: $p(u, v) \approx h(u, v \mid G)$, where $h(u, v \mid G)$ is a predefined heuristic function. Modern methods, however, replace these static heuristics with a learnable function: $p(u, v) = f(u, v \mid G, X, \Theta)$, where X represents node features and Θ denotes the parameters of the model.

PCA. Principal Component Analysis (PCA) is a technique for reducing the dimensionality of data by projecting it onto a lower-dimensional space that captures the directions with the greatest variance in the original data. Let $X \in \mathbb{R}^{N \times D}$ be the input data matrix with N samples and D features, and let Σ be its covariance matrix. PCA provides the best low-rank approximation of the data by minimizing the mean-squared reconstruction error. Let $W \in \mathbb{R}^{D \times K}$ denote the matrix of the top K eigenvectors of Σ. The reconstructed data is given by: $X_{\text{reconstructed}} = (X_{\text{centered}} W)W^\top$, where X_{centered} is the mean-centered version of the input data X. The feature matrix is first centered by subtracting the mean vector $\bar{x} \in \mathbb{R}^D$, computed as $\bar{x}_j = \frac{1}{N}\sum_{i=1}^{N} X_{ij}$, from each feature, resulting in $X_{\text{centered}} = X - 1\bar{x}^\top$, where $1 \in \mathbb{R}^{N \times 1}$ is a column vector of ones. The covariance matrix $\Sigma \in \mathbb{R}^{D \times D}$ is then computed as $\Sigma = \frac{1}{N} X_{\text{centered}}^\top X_{\text{centered}}$. Eigenvalue decomposition of Σ yields $\Sigma = Q \Lambda Q^\top$, where $Q \in \mathbb{R}^{D \times D}$ contains the eigenvectors, and $\Lambda \in \mathbb{R}^{D \times D}$ is a diagonal matrix of eigenvalues, $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D$. By selecting the top K eigenvectors, PCA ensures that the reconstruction error is minimized and that the resulting representation captures the maximum variance of the original data.

Graph Neural Networks. Graph Neural Networks (GNNs) are designed to learn meaningful representations of nodes, edges, or subgraphs by aggregating and transforming local graph information. Formally, given a graph $G = (V, E, A, X)$, the forward propagation in GNNs can be defined as:

$$H^{(l+1)} = \sigma\left(AH^{(l)}W^{(l)}\right), \tag{1}$$

where A is the graph adjacency matrix, $H^{(0)} = X$, representing the initial node features, $H^{(l)}$ is the node feature matrix at layer l, $W^{(l)}$ is the learnable weight matrix, and $\sigma(\cdot)$ is a nonlinear activation function. The resulting node embeddings, $H^{(L)}$, are utilized for subsequent downstream applications.

4. Proposed method

We propose PWLP, a subgraph-based framework for LP that leverages flexible subgraph extraction through random walks, edge-centric encoding via line graph transformation, and balanced feature fusion to support both structural and node-level signals. The overall architecture is illustrated in Figure 2.

4.1. Flexible subgraph extraction

Given a graph $G = (V, E)$ and a target node pair $(u, v) \in V \times V$, PWLP constructs a compact subgraph $G_{u,v}$ that reflects both local and global topological patterns. Unlike conventional methods that extract fixed-radius h-hop neighborhoods, we perform stochastic random walks to prioritize semantically important nodes around the target pair. Specifically, we launch k random walks of length L from each target node u and v. Let $\mathcal{R}_u^r$ denote the set of nodes visited during the r-th walk from node u, and similarly $\mathcal{R}_v^r$ from node v. The union of all visited nodes is:

$$\mathcal{R}_{u,v} = \bigcup_{r=1}^{k} \left(\mathcal{R}_u^r \cup \mathcal{R}_v^r\right), \tag{2}$$

Each node $n \in \mathcal{R}_{u,v}$ is assigned an influence score based on the frequency of visits:

$$s(n) = \sum_{r=1}^{k} \left[I_{n\in\mathcal{R}_u^r} + I_{n\in\mathcal{R}_v^r}\right]. \tag{3}$$

where $I_{\{\cdot\}}$ denotes the indicator function, returning 1 if the condition is true and 0 otherwise. We then select the top-n nodes with the highest scores to form the subgraph:

$$\mathcal{S}_{u,v} = \{n \in \mathcal{R}_{u,v} \mid s(n) \text{ is among top-}n\}. \tag{4}$$

The induced subgraph is:

$$G_{u,v} = (\mathcal{S}_{u,v}, E_{u,v}), \quad E_{u,v} = \{(i, j) \in E \mid i, j \in \mathcal{S}_{u,v}\}. \tag{5}$$

This approach ensures scalability and relevance by dynamically adapting to node influence and graph sparsity.

4.2. Line graph encoding

To model pairwise interactions more effectively, we convert each extracted subgraph $G_{u,v}$ into its corresponding line graph $L(G_{u,v})$, where each node in the line graph corresponds to an edge in the original subgraph. Two nodes in $L(G_{u,v})$ are connected if their corresponding edges share a node in $G_{u,v}$.

Each node $i \in \mathcal{S}_{u,v}$ is assigned a label based on its relative distance to u and v:

$$\begin{aligned} h(i) = 1 &+ \min(d(i, u), d(i, v)) \\ &+ \left\lfloor \frac{d(i,u)}{2} \right\rfloor \cdot \left[\left\lfloor \frac{d(i,u)}{2} \right\rfloor + (d(i, v) \bmod 2) - 1 \right], \end{aligned} \tag{6}$$

where $d(i, u)$ and $d(i, v)$ denote shortest path distances within $G_{u,v}$. For each edge $(i, j) \in E_{u,v}$, its feature vector in the line graph is:

$$x_{L(i,j)} = \text{Concat}\left(\text{one-hot}(h(i)), \text{one-hot}(h(j))\right). \tag{7}$$

Let X_L and A_L denote the feature and adjacency matrices of $L(G_{u,v})$. A GNN encoder f_θ is applied:

$$Z_L = f_\theta(X_L, A_L). \tag{8}$$

The structural embedding of the target edge (u, v) is:

$$SFI_{(u,v)} = Z_L[\text{index}(u, v)]. \tag{9}$$

4.3. Node feature processing

In attributed graphs, node features $X \in \mathbb{R}^{|V|\times D}$ can dominate structural features when $D \gg d$, leading to performance degradation. To prevent this, we apply PCA to reduce dimensionality:

$$\tilde{X} = \text{PCA}(X), \quad \tilde{X} \in \mathbb{R}^{|V|\times d}, \quad d \ll D. \tag{10}$$

We define a node feature interaction (NFI) embedding for the target pair as:

$$\begin{aligned} NFI_{(u,v)} = &\tilde{X}(u) \odot \tilde{X}(v) \\ &|| \ \sigma\left(\tilde{X}(N(u))\right) \odot \sigma\left(\tilde{X}(N(v))\right), \end{aligned} \tag{11}$$

where $\odot$ is the Hadamard product, $||$ denotes concatenation, and $\sigma(\cdot)$ is mean aggregation over a node's neighbors in the subgraph.

4.4. Final prediction.

The final link embedding is obtained by concatenating the structural and node feature representations:

$$e_{(u,v)} = SFI_{(u,v)} \ || \ NFI_{(u,v)}. \tag{12}$$

We use a two-layer MLP to predict the existence of a link:

$$\hat{y}_{(u,v)} = \text{MLP}(e_{(u,v)}). \tag{13}$$

The model is trained using binary cross-entropy loss:

$$\mathcal{L} = -\sum_{(u,v)\in D} y_{(u,v)} \log \hat{y}_{(u,v)} + (1 - y_{(u,v)}) \log(1 - \hat{y}_{(u,v)}), \tag{14}$$

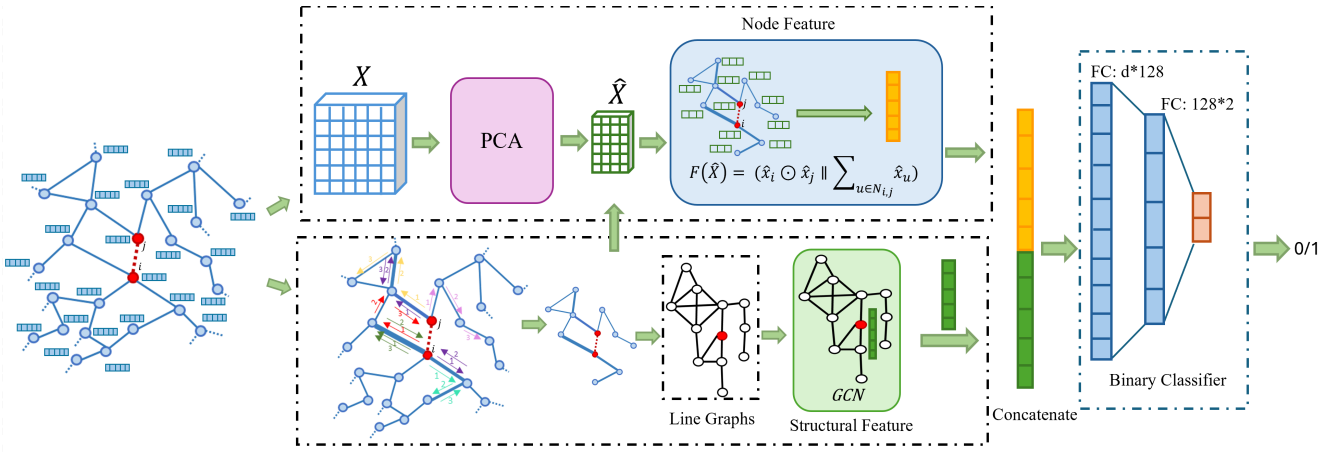


Figure 2: Overview of the PWLP pipeline. Key stages include (1) random walk-based subgraph construction, (2) edge-centric encoding via line graph transformation, (3) PCA-based node feature dimensionality reduction, and (4) final classification.

where $y_{(u,v)} \in \{0, 1\}$ is the true label and $\mathcal{D}$ is the set of training pairs.

The overall procedure of the proposed framework is summarised in Algorithm 1, which outlines the key steps of random walk-based subgraph construction, line graph transformation, structural and node feature encoding, and final link prediction.

5. Theoretical Analysis

We provide theoretical insights into the scalability and expressivity of PWLP. The following Lemmas characterize the efficiency of its subgraph extraction and the ability of edge-centric representation to distinguish graph structure. We also present a corollary that supports our design choice for balanced feature fusion.

Lemma 1. *For any node pair (u, v), the expected size of the subgraph $|V_{u,v}|$ extracted by the PWLP strategy using k random walks of length L is $\mathcal{O}(kL)$. By selecting the top-n most influential nodes, the final subgraph size is further bounded by $\mathcal{O}(n)$, independent of the global graph size $|V|$.*

PROOF. Each random walk from u or v visits at most L nodes. For k walks from each, this yields at most $2kL$ visits. Although nodes may be visited multiple times, the total number of unique visited nodes is upper-bounded by:

$$\mathbb{E}[|\mathcal{R}_{u,v}|] \leq 2kL. \tag{15}$$

We compute the frequency of visits and retain only the top-n most visited nodes. Hence, the subgraph size satisfies:

$$|V_{u,v}| = |\mathcal{S}_{u,v}| \leq n, \tag{16}$$

which is independent of the original graph size $|V|$. This ensures constant-sized subgraphs suitable for scalable training.

Lemma 2. *Let $G_{u,v}$ be the extracted subgraph for a node pair (u, v), and let $L(G_{u,v})$ be its line graph. Under a Weisfeiler-Lehman (WL)-equivalent GNN applied to $L(G_{u,v})$,*

Algorithm 1: PWLP: Pool-Walk Subgraph-Based Link Prediction

Input: Graph $G = (V, E)$, node features $\mathbf{X} \in \mathbb{R}^{|V| \times D}$, target node pair (u, v), walk parameters k, L, subgraph size n
Output: Predicted link probability $\hat{y}_{(u,v)}$
Apply PCA to node features: $\tilde{\mathbf{X}} \leftarrow \text{PCA}(\mathbf{X})$;
Compute NFI: $\mathbf{NFI}_{(u,v)} = \tilde{\mathbf{X}}(u) \odot \tilde{\mathbf{X}}(v) \,||\, \sigma(\tilde{\mathbf{X}}(N(u))) \odot \sigma(\tilde{\mathbf{X}}(N(v)))$;
Initialize visit count map $s(n) = 0$ for all $n \in V$;
foreach $i \in \{u, v\}$ **do**
  for $r = 1$ **to** k **do**
    Perform a random walk of length L from i and record visited nodes $\mathcal{R}_i^r$;
    foreach $n \in \mathcal{R}_i^r$ **do**
      $s(n) \leftarrow s(n) + 1$;
$\mathcal{S}_{u,v} \leftarrow$ Top-n nodes with highest $s(n)$;
$G_{u,v} \leftarrow \text{InducedSubgraph}(G, \mathcal{S}_{u,v})$;
Build line graph $L(G_{u,v})$;
foreach *edge* $(i, j) \in G_{u,v}$ **do**
  Compute distance-based labels $h(i)$ and $h(j)$;
  Set line-graph feature $\mathbf{x}_{L(i,j)} = \text{Concat}(\text{one-hot}(h(i)), \text{one-hot}(h(j)))$;
Apply GNN f_θ on $L(G_{u,v})$ to obtain $\mathbf{Z}_L$;
$\mathbf{SFI}_{(u,v)} \leftarrow \mathbf{Z}_L[\text{index}(u, v)]$;
Fuse features: $\mathbf{e}_{(u,v)} = \mathbf{SFI}_{(u,v)} || \mathbf{NFI}_{(u,v)}$;
Predict link probability: $\hat{y}_{(u,v)} = \text{MLP}(\mathbf{e}_{(u,v)})$;
return $\hat{y}_{(u,v)}$;

PWLP can distinguish any two non-isomorphic edge-centric subgraphs of G that differ in local structure around the target edge.

PROOF. Consider two subgraphs $G_{u,v}^1$ and $G_{u,v}^2$ that differ in edge structure. The line graph transformation maps each edge of the original subgraph to a node in the line graph.

Node features in $L(G_{u,v})$ are derived from distance-aware labels with one-hot encodings that encode relational context. Since the GNN used is WL-equivalent, and the initial features distinguish non-isomorphic structures, the final embeddings Z_L will also differ. Thus, PWLP can distinguish such subgraphs.

Corollary 3. *Let $SFI_{(i,j)} \in \mathbb{R}^{d_s}$ and $NFI_{(i,j)} \in \mathbb{R}^{d_n}$ be the structural and node feature embeddings. When $d_n \approx d_s$, the fused embedding $e(i,j) = Concat(SFI_{(i,j)}, NFI_{(i,j)})$ leads to more stable gradient updates and reduces modality dominance.*

PROOF. Assume the original node features $X \in \mathbb{R}^{N\times D}$, with $D \gg d_s$. Concatenating high-dimensional node features with shorter structural embeddings can result in over-representation of node-based information. Applying PCA transforms $X \to \tilde{X} \in \mathbb{R}^{N\times d_n}$, where $d_n \approx d_s$. This alignment ensures that gradient updates during backpropagation are balanced across feature types, promoting stable learning and improved generalization.

6. Experiments

6.1. Datasets

The methods are evaluated on ten datasets, including three citation networks (*Cora*, *Citeseer*, and *Pubmed*) from the Planetoid collection Yang et al. (2016), the large-scale *OGBL-Collab* benchmark from the Open Graph Benchmark (OGB), and six additional graphs (*Router*, *ADV*, *SMG*, *HPD*, *EML*, and *KHN*).[1] A brief overview of the benchmark datasets is provided in table 1. These datasets span diverse graph structures and domains (e.g., social, citation, and biological networks), offering a comprehensive basis for assessing model performance.

6.2. Baseline Methods

This section provides detailed descriptions of the baseline methods utilized in this paper. In this section, we evaluate the performance of PWLP against a diverse set of baselines, grouped into two categories: heuristic methods, including Common Neighbors (CN), Adamic-Adar (AA), Shortest Path (SP), and Katz; and learning based methods, including SEAL, LGLP, NBFNet, Neo-GNN, BUDDY, PEG, and NCNC across a diverse range of graph families and problem settings. Table 2 shows Details of heuristic-based methods utilized in this paper.

6.3. Experimental Setup

This section provides details of our experimental setup to support full reproducibility. All datasets were randomly split into 70% training, 10% validation, and 20% testing sets, except for OGBL-Collab, for which we used the predefined train/validation/test split provided by the Open Graph Benchmark to ensure consistency with prior work. All learning-based models were trained for 50 epochs with a batch size of 50. Experiments were repeated 10 times, and the results are reported as the mean ± standard deviation to reflect reliability. The damping factor for Katz was set to 0.05. To ensure a fair comparison, particularly with SEAL, LGLP, PEG, and NCNC, all parameters were configured according to their original papers. Each of the three graph convolution layers used an output feature dimension of 32. We employed the Adam optimizer with a learning rate of 5e-3. All experiments were conducted on AWS EC2 `ml.p3.2xlarge` instances, each equipped with one NVIDIA V100 GPU, 8 vCPUs, and 61 GB of RAM.

[1] https://noesis.ikor.org/datasets/link-prediction

Table 1
Summary Statistics of the Datasets Used in the Study.

DATA SET	#NODES	#EDGES	#FEAT.	TYPE
OGBL-COLLAB	235,868	1,285,465	128	COLLABORATION
CORA	2,708	5,278	1,433	CITATION NETWORK
CITESEER	3,327	4,552	3,703	CITATION NETWORK
PUBMED	19,717	44,324	500	CITATION NETWORK
ROUTER	5,022	6,258	×	NETWORK TOPOLOGY
ADV	5,155	39,285	×	SOCIAL NETWORK
SMG	1,024	4,916	×	CO-AUTHORSHIP
HPD	8,756	32,331	×	BIOLOGY
EML	1,133	5,451	×	SHARED EMAILS
KHN	3,772	12,718	×	CO-AUTHORSHIP

Table 2
Details of heuristic-based methods utilized in this paper.

Name	Formula	Order
COMMON NEIGHBORS	$\lvert\Gamma(x) \cap \Gamma(y)\rvert$	FIRST
ADAMIC-ADAR	$\sum_{z\in\Gamma(x)\cap\Gamma(y)} \frac{1}{\log\lvert\Gamma(z)\rvert}$	SECOND
RESOURCE ALLOCATION	$\sum_{z\in\Gamma(x)\cap\Gamma(y)} \frac{1}{\lvert\Gamma(z)\rvert}$	SECOND
SHORTEST PATH	$\frac{1}{length(shortestpath(x,y)}$	HIGH
KATZ	$\sum_{l=1}^{\infty} \beta^l \lvert\text{WALKS}^{\langle l\rangle}(x,y)\rvert$	HIGH

6.4. Evaluation Metrics

This section describes the evaluation metrics used, including AUC, AP, and Hit@K. The Area Under the Curve (AUC) is calculated as the proportion of correct predictions out of all comparisons made. A correct prediction occurs when a positive sample scores higher than a negative sample based on heuristic scores (e.g., common neighbors) or probabilities generated by the GNN model. The AUC is computed using the formula:

$$AUC = \frac{n' + 0.5 \times n''}{n} \tag{17}$$

where n represents the total number of prediction pairs, n' is the count of pairs where the positive sample has a higher score, and n'' counts ties where both samples have equal scores. Average Precision (AP) evaluates the model's precision across different threshold levels, reflecting how well positive links are ranked above negative ones. It is defined as:

$$AP = \sum_{k=1}^{n} P(k) \cdot \Delta r(k) \tag{18}$$

where n is the total number of positive and negative samples, $P(k)$ is the precision at position k, and $\Delta r(k)$ is the change in recall at position k. Hit@K (also known as Recall@K) measures the proportion of times a true positive item appears within the top K predictions made by the model. For each query, a "hit" occurs if the true positive is ranked among the top K results. The Hit@K score is calculated as:

$$\text{Hit@K} = \frac{\text{Number of hits in top } K}{\text{Total number of queries}} \tag{19}$$

A higher Hit@K indicates better performance in retrieving relevant links within the top K predictions.

6.5. Motivational Observations

Local heuristics such as CN, AA, and RA rely primarily on one-hop neighborhood overlaps, capturing only shallow structural cues. As shown in Figure 3, their performance remains modest across datasets, since they treat shared neighbors uniformly and ignore the influence of multi-hop connectors. In contrast, Katz, which aggregates contributions from paths of multiple lengths, achieves significantly higher AP scores, highlighting the predictive value of long-range structural dependencies.

These observations suggest that local information alone is insufficient for reliable link prediction. This motivates the design of PWLP, which extracts random-walk-based subgraphs to capture long-range connectivity and leverage these multi-hop signals within an edge-centric representation.

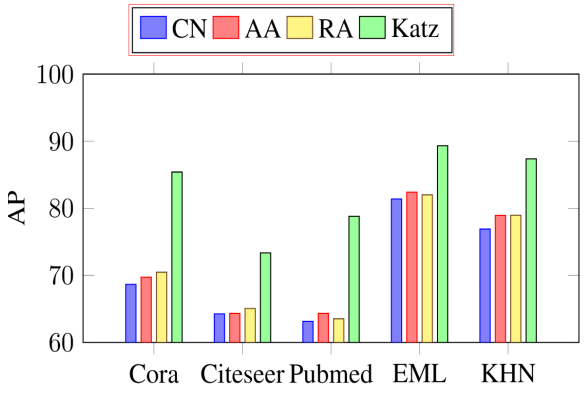


Figure 3: Effect of multi-hop neighbor information on AP scores using different heuristic methods (CN, AA, RA, and Katz) across the Cora, Citeseer, Pubmed, EML, and KHN datasets.

6.6. Effectiveness of Node Features

Table 3 highlights an important limitation of existing methods, when node features are either present or unavailable. Interestingly, methods like LGLP and SEAL, which integrate structural features with MPNNs, exhibit a performance drop when node features are included. For example, in the Cora dataset, SEAL achieves an AP score of 94.84 with structural features, but its performance decreases to 92.31 when node features are used. Similarly, LGLP shows a performance decline from 96.13 (with structural features) to 92.14 (with node features). This observation suggests that the addition of node features in these methods might introduce redundancy, negatively affecting their ability to leverage structural information effectively. On the other hand, methods like NCNC and PEG, which heavily rely on node features in message-passing mechanisms, experience significant performance degradation when node features are unavailable or replaced by random features or degree-based features. For instance, NCNC's performance on the Cora dataset drops from 96.08 (with node features) to 83.84 (with random features), underscoring its dependency on high-quality node features. This contrast highlights the trade-offs in graph learning approaches: while structural features can enhance the expressivity and robustness of methods like SEAL and LGLP, excessive reliance on node features in methods like NCNC and PEG may reduce their effectiveness in scenarios where such features are unavailable or insufficient. PWLP exhibits superior performance with or without node features, and notably, its performance improves when node features are incorporated. As shown in Table 3, on the Cora dataset, PWLP achieves an AP score of 96.22 with structural features and slightly improves to 96.44 when node features are added. Similar trends are observed for the Citeseer and Pubmed datasets, where PWLP maintains high scores, demonstrating its ability to effectively utilize node features to enhance its predictive capability. This consistent improvement with the addition of node features underscores PWLP's flexibility and robustness. Unlike other methods, such as SEAL and LGLP, which suffer from performance drops when incorporating node features, PWLP not only avoids such pitfalls but also capitalizes on the additional information provided by node features to deliver better results. This adaptability makes PWLP a highly reliable and versatile method for link prediction tasks across various datasets.

6.7. Overall Performance on Benchmark Datasets

Table 4 reports the Average Precision (AP) scores for six non-attributed datasets, including *Router*, *ADV*, *SMG*, *HPD*, *EML*, and *KHN*, under a 0.2 test split ratio. Traditional heuristics such as CN, AA, and RA deliver moderate performance but are consistently surpassed by more advanced approaches. Among heuristic baselines, Katz and Shortest Path yield comparatively stronger results, particularly on denser graphs such as *ADV* and *EML*.

Subgraph-based models, including SEAL, LGLP, and PWLP, achieve the highest AP scores across nearly all datasets, reflecting their superior ability to encode localized structural dependencies, even in the absence of node features. Notably, PWLP attains the best overall performance, marginally outperforming LGLP and SEAL while benefiting from smaller enclosing subgraphs and more efficient inference. In contrast, methods such as LGLP and SEAL require larger subgraphs, which leads to higher computational overhead. PEG and NCNC, which rely on global structural information, remain competitive but generally fall short of subgraph-based models. These results highlight both the

Table 3
Impact of node features on AP performance of NCN, PEG, LGLP, SEAL, and PWLP.

	F	Cora	Citeseer	Pubmed
SEAL	NODE FEATURE	92.31±0.19	94.68±0.01	97.92±0.06
	STRUCTURAL FEATURE	94.84±0.02	94.82±0.04	97.32±0.00
LGLP	NODE FEATURE	92.14±0.18	91.85±0.02	97.04±0.22
	STRUCTURAL FEATURE	96.13±0.07	95.76±0.13	98.43±0.20
PEG	NODE FEATURE	87.98±1.40	89.55±1.17	77.34±5.60
	RANDOM FEATURE	85.93±0.79	83.55±1.63	86.34±0.50
NCNC	NODE FEATURE	96.08±0.42	97.40±0.37	**98.75±0.11**
	RANDOM FEATURE	83.84±0.73	78.10±0.91	93.97±0.73
	DEGREE-BASED FEATURE	86.83±1.31	85.68±0.63	95.18±0.13
PWLP	NODE FEATURE	**96.56±0.09**	**97.46±0.42**	98.53±0.11
	STRUCTURAL FEATURE	96.22±0.30	95.60±0.57	98.28±0.09

Table 4
Average precision (AP) of baseline methods on non-attributed datasets (test ratio = 0.2).

Method	Router	ADV	SMG	HPD	EML	KHN
CN	55.14±0.37	87.98±0.27	81.00±0.74	71.50±0.17	81.39±0.81	76.90±0.37
AA	55.21±0.32	89.01±0.29	83.77±0.78	71.84±0.19	82.40±0.87	78.94±0.31
RA	55.18±0.33	89.03±0.29	83.79±0.61	71.80±0.20	82.29±0.83	78.96±0.29
SP	62.77±0.42	84.90±0.09	76.40±0.46	84.45±0.16	83.25±1.08	80.54±0.26
Katz	62.95±0.31	93.43±0.13	88.29±0.72	86.40±0.19	89.32±0.64	87.37±0.30
SEAL	97.15±0.00	96.77±0.02	90.87±0.03	95.38±0.02	92.93±0.03	94.81±0.02
LGLP	99.15±0.14	96.77±0.06	93.76±0.35	95.98±0.15	94.16±0.13	96.08±0.23
PEG	80.25±0.69	92.55±0.36	86.46±0.44	88.57±0.61	87.80±0.58	86.60±6.63
NCNC	84.50±1.00	91.61±1.81	88.45±1.45	86.22±0.44	83.68±4.73	87.73±2.15
PWLP	**99.41±0.15**	**96.93±0.06**	**94.04±0.65**	**96.25±0.04**	**94.19±0.30**	**96.14±0.08**

robustness and efficiency of subgraph-oriented learning in non-attributed scenarios.

To complement the AP-based evaluation, Table 5 presents the AUC results on the same datasets under identical settings. PWLP consistently ranks first across most datasets, demonstrating strong generalization and robustness to diverse structural patterns. For instance, on the challenging KHN dataset, PWLP attains an AUC of 95.66 ± 0.15, outperforming the strongest heuristic method (Katz: 86.42 ± 0.34) and the best neural baseline (PEG: 85.72 ± 3.71). Similar performance gaps are observed on HPD, EML, and ADV, indicating that PWLP effectively captures both local and global structural cues that are critical for link formation.

While heuristics exhibit competitive behavior in simpler graphs, they struggle to model complex relational patterns due to their limited expressiveness. Neural baselines such as SEAL, LGLP, and PEG demonstrate stronger performance overall, but their scores vary across datasets. In contrast, PWLP not only achieves higher accuracy but also maintains low variance, underscoring the benefit of its distance-aware selective aggregation and lightweight subgraph refinement strategy.

The comparative analysis clearly shows that: (i) subgraph-based techniques offer the strongest predictive capability in non-attributed settings; (ii) PWLP delivers state-of-the-art performance with improved efficiency; and (iii) reliance solely on global heuristics or shallow structural cues is insufficient for complex topologies. Overall, PWLP demonstrates a strong balance between accuracy, robustness, and computational efficiency.

To further demonstrate the effectiveness of PWLP, we report the Hit@K results on the Cora, Pubmed, and OGBL-Collab datasets. As summarized in Table 6, PWLP achieves the highest Hit@100 scores on both Cora and Pubmed, and the highest Hit@50 score on the large-scale OGBL-Collab benchmark, outperforming recent graph-based predictors such as NCNC, Buddy, SEAL, LGLP, Neo-GNN, and NBFNet. These results indicate that PWLP can accurately prioritize structurally relevant neighbors and effectively rank candidate links across both small citation networks and large-scale real-world graphs.

Finally, to verify the statistical significance of our improvements, we conduct two-sided paired t-tests ($\alpha = 0.05$) comparing PWLP with strong baselines on the Cora and HPD datasets. As reported in Table 7, all p-values are below 0.05, indicating that the improvements of PWLP are statistically significant rather than due to random variation. This provides strong evidence that the performance gains

Table 5
AUCs on 14 datasets for all baseline methods for test-ratio = 0.2

	Router	ADV	SMG	HPD	EML	KHN
CN	55.16±0.36	88.36±0.28	82.21±0.74	71.65±0.16	82.05±0.78	77.57±0.35
AA	55.16±0.36	88.67±0.29	83.08±0.74	71.70±0.16	82.23±0.83	78.14±0.34
RA	55.16±0.36	88.69±0.29	83.11±0.68	71.69±0.17	82.19±0.84	78.15±0.33
SP	43.17±0.60	88.04±0.06	80.99±0.38	83.71±0.26	85.77±1.03	82.07±0.29
Katz	62.82±0.26	92.57±0.09	87.24±0.59	85.95±0.19	88.79±0.66	86.42±0.34
SEAL	97.19±0.00	96.60±0.02	91.23±0.03	94.73±0.00	92.58±0.06	94.31±0.01
LGLP	99.04±0.13	96.71±0.09	93.75±0.36	95.24±0.16	93.82±0.17	95.43±0.28
PEG	79.23±0.66	91.96±0.39	84.87±0.66	87.08±0.39	87.36±0.26	85.72±3.71
NCNC	79.33±1.68	90.70±1.20	85.70±1.56	63.88±11.5	84.57±2.50	82.07±7.50
PWLP	**99.26±0.12**	**96.86±0.01**	**93.82±0.38**	**95.83±0.04**	**94.22±0.32**	**95.66±0.15**

Table 6
Comparison of Hit@K performance on the Cora, Pubmed, and OGBL-Collab datasets.

METHOD	CORA HIT@100	PUBMED HIT@100	OGBL-COLLAB HIT@50
NCNC	89.19±1.36	81.53±0.95	66.72±0.87
BUDDY	88.01±0.64	74.10±0.78	65.87±0.58
SEAL	80.28±1.31	76.33±1.32	64.95±0.43
LGLP	88.44±1.12	83.57±1.04	OOM
NBFNET	71.65±2.17	74.07±1.99	OOM
NEO-GNN	80.42±1.29	73.93±1.19	57.81±0.37
PWLP	**91.18±0.69**	**85.11±0.81**	**68.74±0.92**

Table 7
Paired two-sided t-test results ($\alpha = 0.05$) comparing PWLP with LGLP and NCNC.

DATASET	PWLP VS LGLP	PWLP VS NCNC
CORA	9.73×10^{-3}	4.98×10^{-2}
HPD	3.57×10^{-3}	5.02×10^{-3}

achieved by PWLP are consistent and reliable across multiple datasets.

6.8. AUC and Loss Convergence Curves

Figure 4 presents the training AUC curves over 50 epochs for PWLP, LGLP, and SEAL across nine benchmark datasets. PWLP consistently achieves superior performance, exhibiting higher final AUC values across different scenarios. Specifically, PWLP's distance-aware selective aggregation enables it to prioritize the most relevant neighbors, allowing for more efficient learning of complex structural patterns. The results support our claim that leveraging multi-hop distance information while maintaining a compact and relevant subgraph leads to better training efficiency and higher predictive performance

Figure 5 shows the training loss curves over 50 epochs for PWLP, LGLP, and SEAL across nine datasets. As depicted, PWLP consistently achieves the lowest training loss throughout the training process, with a notably faster and more stable convergence compared to the baselines. This indicates that PWLP optimizes the objective function more effectively, reflecting its ability to capture useful structural and positional features for link prediction.

The performance gap is particularly evident on datasets such as Cora, Citeseer, SMG, and Router, where SEAL suffers from relatively high and unstable loss, and LGLP plateaus early. In contrast, PWLP continues to refine its predictions throughout training, suggesting better generalization and resistance to overfitting. These findings reinforce the advantage of PWLP's distance-aware selective aggregation in learning compact yet informative subgraph representations that guide the model toward faster and more accurate optimization

6.9. Ablation Study

6.9.1. Impact of Top-N Node Selection

We conduct a comparative analysis on the EML, Cora, and ADV datasets by varying the number of selected top nodes (Top-$N \in \{5, 10, 25, 50, 100, 150, 200\}$). Reducing Top-$N$ yields smaller subgraphs with fewer nodes and edges, which directly translates into lower CUDA memory consumption and faster processing times. Conversely, increasing Top-N enhances the model's capacity to capture richer structural patterns, improving predictive performance in tasks that require detailed subgraph understanding. As shown in Figure 6, performance improves substantially when increasing the number of nodes from very small values (e.g., 5 to 25). However, beyond approximately 50 nodes, the performance plateaus, indicating a saturation point. This suggests that selecting fewer than 50 nodes per subgraph is sufficient to achieve performance comparable to LGLP, striking a desirable balance between efficiency and accuracy.

6.9.2. Effect of Top-N Node Selection on Hit@k

To further examine this behavior, we analyze the effect of Top-N on Hit@k across the EML, Cora, and ADV datasets. As illustrated in Figure 7, PWLP achieves competitive performance even with a small number of selected nodes. Remarkably, with only 25 nodes in the subgraph, PWLP attains results comparable to much larger configurations. When using 50 nodes, PWLP not only maintains efficiency but also surpasses LGLP, highlighting the scalability and

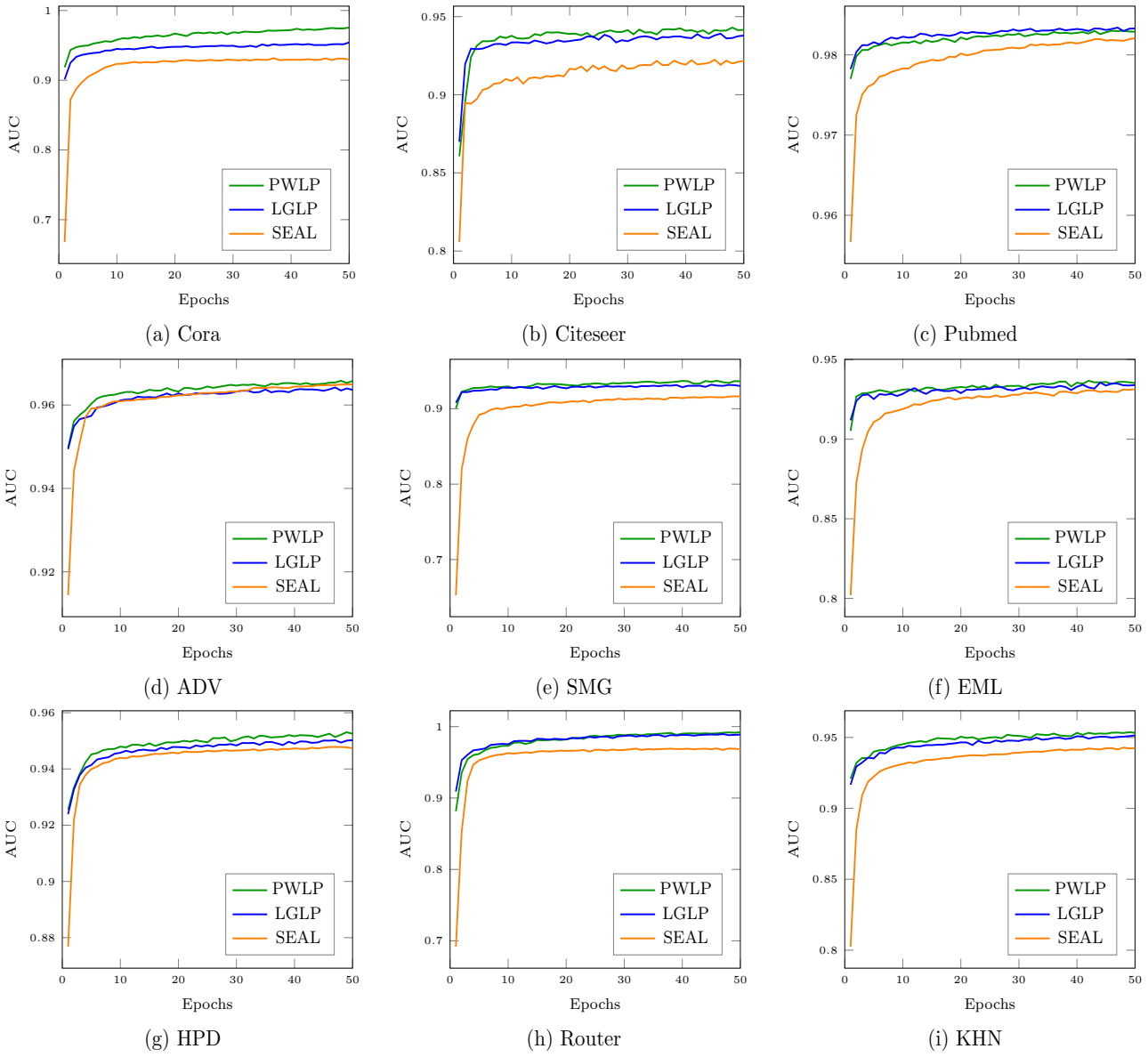


Figure 4: Comparison of training AUCs for PWLP, LGLP, and SEAL models.

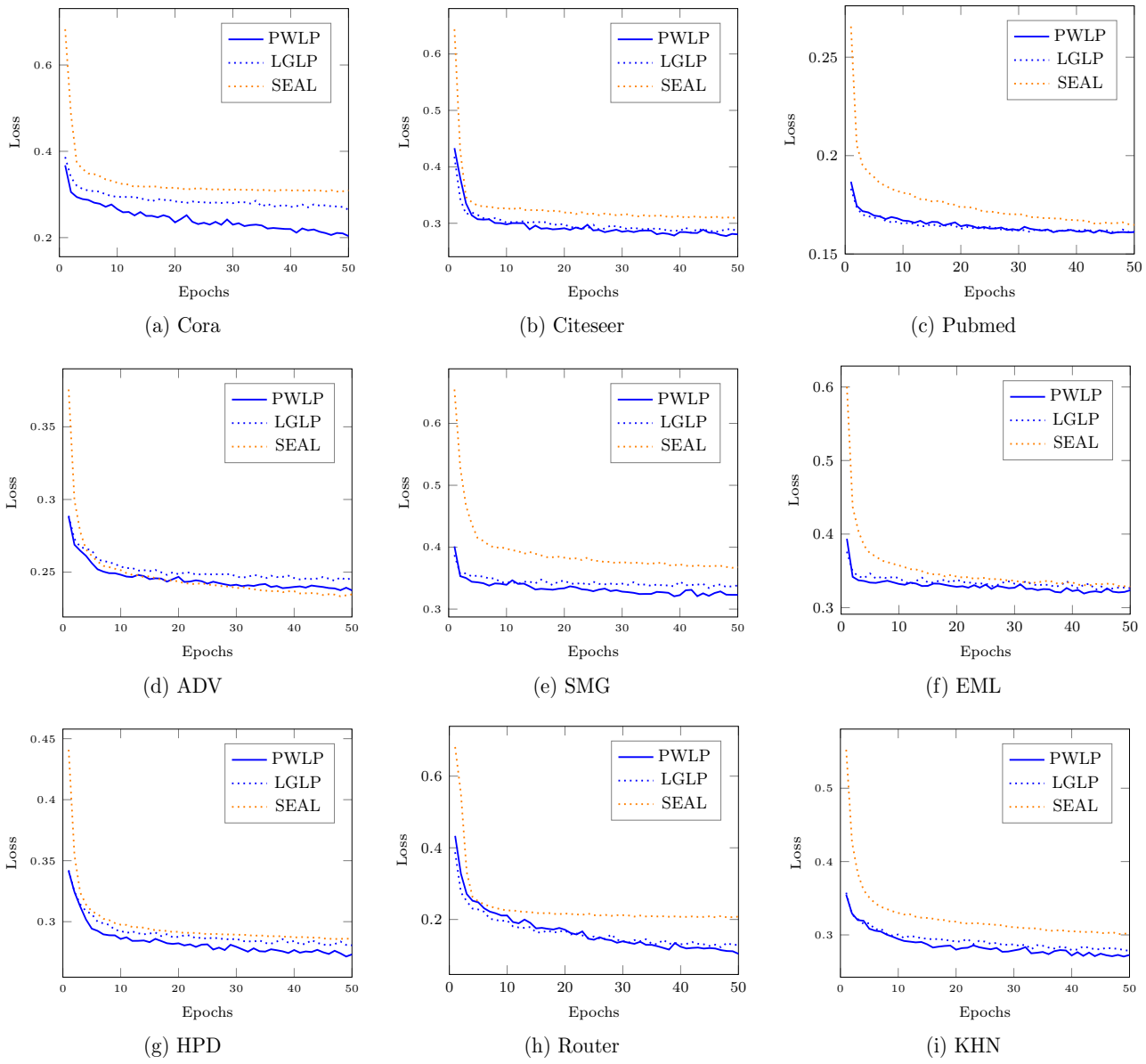
effectiveness of selective node expansion in capturing task-relevant structural information.

6.9.3. Effect of Walk Length and Number of Walks

In this section, we conduct ablation studies on PWLP, focusing on length of walks and the number of walks. A further analysis on the EML dataset assessed how different numbers of walks and walk lengths affect performance. Comparative results were measured using AUC, AP, and Hit@100. Fewer walks or shorter walk lengths reduce the computational overhead and CUDA resource usage. More walks or longer walk lengths capture higher-order relationships within the graph, which can boost performance metrics, but at increased computational cost. Figure 8 shows results on different configurations of walk parameters (e.g., 5 vs. 10 vs. 25 vs. 50 walks, and 2,3,5,7 lengths) are plotted against the AUC, AP, and Hit@100 scores.

6.9.4. Robustness to Noisy or Missing Features

To evaluate the robustness of PWLP against noisy or missing node attributes, we replace all node features with random vectors and compare the performance with the original feature setting. As reported in Table 8, PWLP maintains high predictive performance under extreme noise, whereas feature-dependent baselines such as NCNC suffer a significant degradation. On Cora and Pubmed, PWLP drops only marginally in both AP and Hit@100, confirming that our structural design enables the model to remain stable even when node features are unreliable or corrupted.

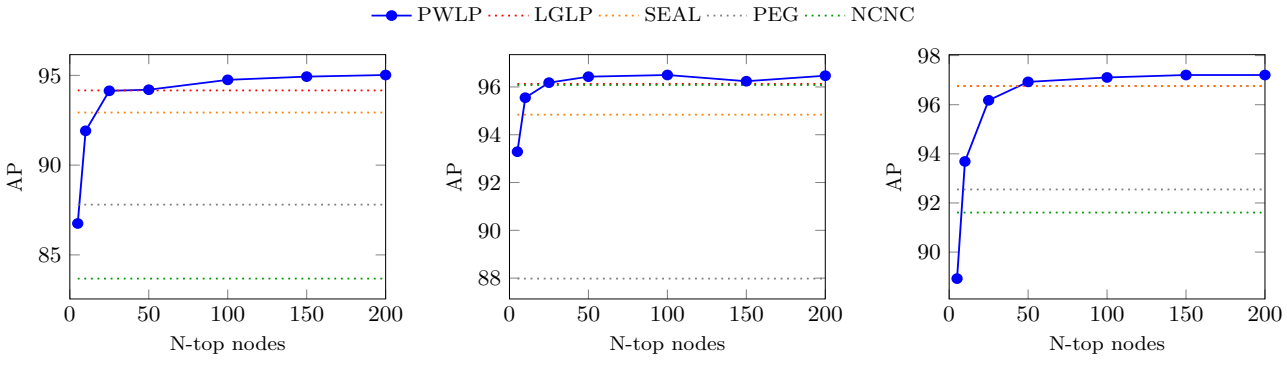


(a) Cora (b) Citeseer (c) Pubmed

(d) ADV (e) SMG (f) EML

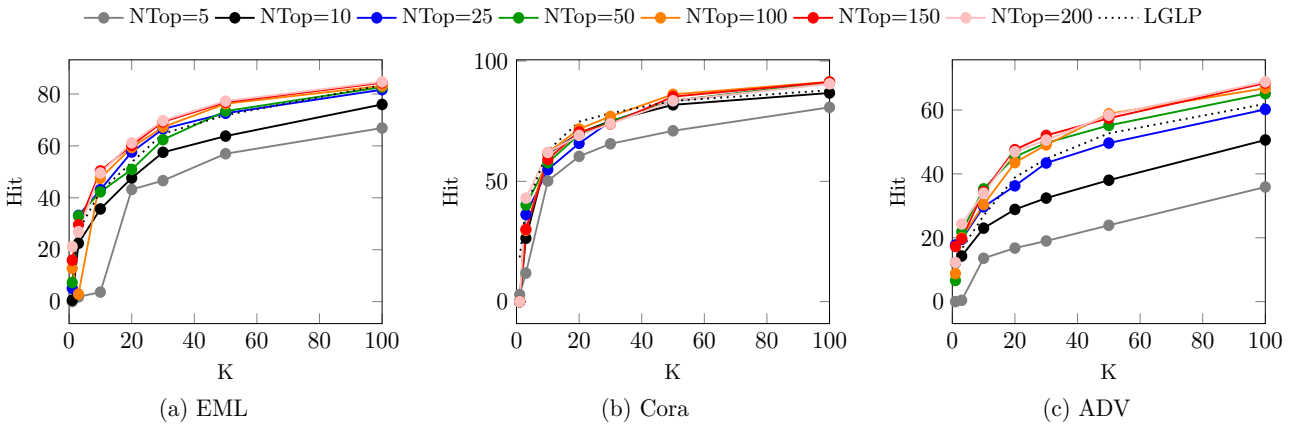
(g) HPD (h) Router (i) KHN

Figure 5: Comparison of training losses for PWLP, LGLP, and SEAL models.



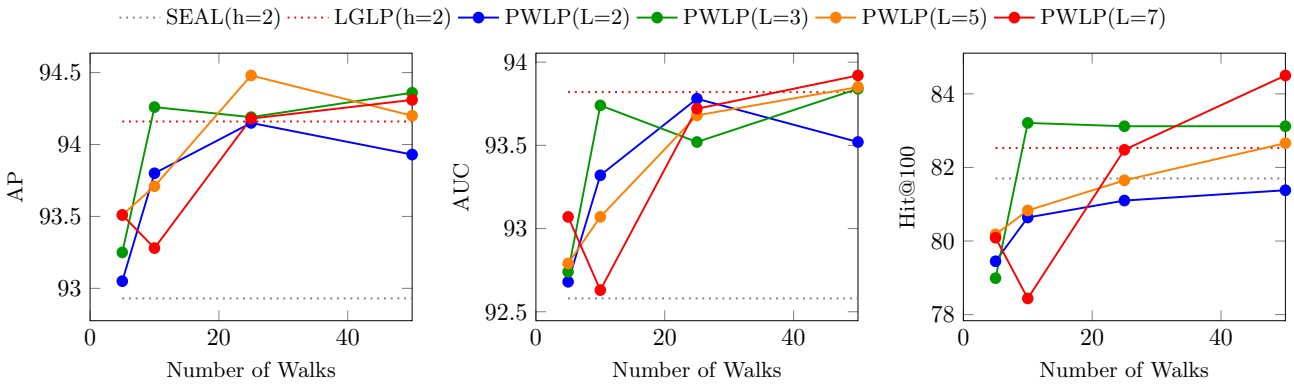
(a) EML (b) Cora (c) ADV

Figure 6: Impact of varying the number of selected top nodes (N-top) on the EML, Cora, and ADV datasets.



(a) EML (b) Cora (c) ADV

Figure 7: Hit performance at varying N-top node rates compared to the LGLP baseline.



(a) AP on EML (b) AUC on EML (c) Hit@100 on EML

Figure 8: Comparative analysis of the EML dataset using AUC, AP, and Hit@100.

Table 8
Robustness to noisy node features on Cora and Pubmed under random feature perturbations.

Method	Hit@100		AP	
	Cora	Pubmed	Cora	Pubmed
NCNC-Random	63.12	55.74	83.84	93.97
NCNC-Node Feature	89.19	81.53	96.08	98.75
PWLP-Random	88.15	84.67	96.12	98.15
PWLP-Node Feature	**91.18**	**85.11**	**96.56**	**98.53**

Table 9
Impact of PCA on the Cora dataset under noisy feature conditions.

Setting	AP	Hit@100
Without PCA	94.91	80.28
With PCA	**96.12**	**88.15**

6.9.5. Effect of PCA

We apply PCA to stabilize and normalize node attributes when they are high-dimensional or noisy. As shown in Table 9, removing PCA causes a clear drop in both AP and Hit@100 on Cora. This confirms that PCA improves compatibility between structural and attribute signals and mitigates noise effects, consistent with observations reported in SEAL and LGLP.

6.10. Time Complexity Analysis

Pre-processing complexity. In SEAL, pre-processing consists of extracting the h-hop enclosing subgraph around each target link and computing structural node labels (e.g., DRNL). This step requires traversing local neighborhoods and computing shortest-path distances, resulting in a complexity of $\mathcal{O}(|E_s|)$, where $|E_s|$ is the number of edges in the extracted subgraph. In practice, this depends on node degree and hop count, but is linear in the subgraph size. LGLP extends SEAL by converting the extracted subgraph into a line graph. This transformation requires examining all pairs of adjacent edges to form connections in the line graph, resulting in $\mathcal{O}(n^2 + m)$ pre-processing complexity, where n and m are the number of nodes and edges in the original subgraph. Since $m \leq \mathcal{O}(n^2)$ for sparse graphs, the overall complexity simplifies to $\mathcal{O}(n^2)$. In contrast, PWLP adopts a lightweight strategy. It initiates k random walks of length w from each of f starting nodes (typically the two target endpoints), producing $\mathcal{O}(kfw)$ total node visits. The top-n most frequently visited nodes are then selected to form

Table 10
Comparison of pre-processing and inference-time complexities.

METHOD	PRE-PROCESSING	INFERENCE
SEAL	$\mathcal{O}(m)$	$\mathcal{O}(\ell m d d' + d'^2)$
LGLP	$\mathcal{O}(n^2)$	$\mathcal{O}(\ell m' d d' + d'^2)$
PWLP	$\mathcal{O}(k f w)$	$\mathcal{O}(\ell d d' + d'^2)$

the subgraph. Since the subgraph size is fixed and small, line graph construction and feature labeling remain bounded. Thus, the overall pre-processing complexity for PWLP is $\mathcal{O}(k f w)$, independent of the size or density of the original graph.

Inference-time complexity. Subgraph-based link prediction methods typically involve two stages: (1) representation learning via message passing (e.g., GNN), and (2) classification using a multilayer perceptron (MLP). In SEAL, an ℓ-layer GNN is applied to the subgraph with $|E_s|$ edges. The complexity of the message-passing phase is $\mathcal{O}(\ell |E_s| d d')$, where d and d' denote the input and hidden feature dimensions. A pooling operation (e.g., SortPooling) adds $\mathcal{O}(nd')$, but is typically negligible compared to the GNN stage. LGLP performs message passing on the line graph derived from the subgraph. Since the line graph has significantly more edges ($|E'_s| \gg |E_s|$), the complexity increases to $\mathcal{O}(\ell |E'_s| d d')$. PWLP, on the other hand, constructs a small fixed-size subgraph (e.g., $n = 10$) and converts it into a compact line graph. As the number of nodes and edges in this line graph is bounded (e.g., $|V'_s|, |E'_s| = \mathcal{O}(n), \mathcal{O}(n^2)$), the complexity of GNN encoding becomes $\mathcal{O}(\ell d d')$, with no dependence on the global graph structure. Following representation learning, each method applies a 2-layer MLP to compute the link score. For an input vector $\mathbf{x} \in \mathbb{R}^d$, the MLP computes: $\text{MLP}(\mathbf{x}) = W_2 \cdot \sigma(W_1 \cdot \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$, where $W_1 \in \mathbb{R}^{d \times d}$, $W_2 \in \mathbb{R}^{2 \times d}$, and σ is a non-linear activation (e.g., ReLU). The dominant cost arises from the first layer, yielding total complexity $\mathcal{O}(d^2)$. Since the MLP is applied once per subgraph, its cost is generally dominated by the GNN encoding step. PWLP achieves significantly more efficient pre-processing and inference than SEAL and LGLP by operating on compact, influence-guided subgraphs. Unlike SEAL and LGLP, which process entire local neighborhoods or large edge-transformed graphs, PWLP's bounded sampling and representation enable constant-time inference per instance, with no dependence on the graph size $|V|$ or $|E|$. The comparative pre-processing and inference-time complexities of SEAL, LGLP, and PWLP are summarized in Table 10.

7. Conclusion

In this work, we introduced PWLP, a novel subgraph-based link prediction framework that effectively balances scalability and structural richness by combining local and global information through a pool-walk strategy. Unlike fixed-hop methods, our approach dynamically selects influential nodes via random walks, enabling compact yet expressive subgraphs that capture both neighborhood context and long-range dependencies. This design significantly reduces inference-time cost and computational overhead compared to the state-of-the-art approaches, while maintaining strong predictive performance. Empirical evaluations across multiple benchmark datasets demonstrate the superiority of PWLP in both accuracy and efficiency, establishing it as a scalable and generalizable solution for link prediction in diverse graph-based applications. In future work, we aim to extend the utility of edge-centric knowledge by applying it to additional graph learning tasks, including node classification and graph reconstruction.

References

Adamic, L.A., Adar, E., 2003. Friends and neighbors on the web. Social Networks 25, 211–230.

Cai, L., Ji, S., 2020. A multi-scale approach for graph link prediction, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 3308–3315.

Cai, L., Li, J., Wang, J., Ji, S., 2021. Line graph neural networks for link prediction. IEEE Transactions on Pattern Analysis and Machine Intelligence 44, 5103–5113.

Chamberlain, B., Shirobokov, S., Rossi, E., Frasca, F., Markovich, T., Hammerla, N., Bronstein, M., Hansmire, M., 2022. Graph neural networks for link prediction with subgraph sketching. arXiv preprint arXiv:2209.15486 .

Hamilton, W.L., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems.

Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., Leskovec, J., 2020. Open graph benchmark: Datasets for machine learning on graphs, in: Advances in Neural Information Processing Systems, pp. 22118–22133.

Katz, L., 1953. A new status index derived from sociometric analysis. Psychometrika 18, 39–43.

Kipf, T.N., Welling, M., 2016. Variational graph auto-encoders. arXiv preprint arXiv:1611.07308 .

Li, P., Wang, Y., Wang, H., Leskovec, J., 2020. Distance encoding: Design provably more powerful neural networks for graph representation learning, in: Advances in Neural Information Processing Systems, pp. 4465–4478.

Louis, P., Jacob, S., Salehi-Abari, A., 2022. Sampling enclosing subgraphs for link prediction, in: Proceedings of the 31st ACM International Conference on Information and Knowledge Management, pp. 4269–4273.

Louis, P., Jacob, S., Salehi-Abari, A., 2023. Simplifying subgraph representation learning for scalable link prediction. arXiv preprint arXiv:2301.12562 .

Menon, A.K., Elkan, C., 2011. Link prediction via matrix factorization, in: Machine Learning and Knowledge Discovery in Databases (ECML PKDD 2011), Proceedings, Part II, pp. 437–452.

Newman, M.E.J., 2001. Clustering and preferential attachment in growing networks. Physical Review E 64, 025102.

Perozzi, B., Al-Rfou, R., Skiena, S., 2014. Deepwalk: Online learning of social representations, in: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 701–710.

Qiu, J., Dong, Y., Ma, H., Li, J., Wang, K., Tang, J., 2018. Network embedding as matrix factorization: Unifying deepwalk, line, pte, and node2vec, in: Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining, pp. 459–467.

Tang, J., Qu, M., Wang, M., Zhang, M., Yan, J., Mei, Q., 2015. Line: Large-scale information network embedding, in: Proceedings of the 24th

International Conference on World Wide Web, pp. 1067–1077.
Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., 2017. Graph attention networks. arXiv preprint arXiv:1710.10903 .
Wang, H., Yin, H., Zhang, M., Li, P., 2022. Equivariant and stable positional encoding for more powerful graph neural networks. arXiv preprint arXiv:2203.00199 .
Wang, X., Yang, H., Zhang, M., 2024. Neural common neighbor with completion for link prediction, in: The Twelfth International Conference on Learning Representations. URL: https://openreview.net/forum?id=sNFLN3itAd.
Yang, Y., Zhang, J., Zhu, X., Tian, L., 2018. Link prediction via significant influence. Physica A: Statistical Mechanics and its Applications 492, 1523–1530.
Yang, Z., Cohen, W.W., Salakhutdinov, R., 2016. Revisiting semi-supervised learning with graph embeddings, in: Proceedings of the 33rd International Conference on Machine Learning, pp. 40–48.
Yao, L., Mao, C., Luo, Y., 2019. Graph convolutional networks for text classification, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 7370–7377.
Yin, H., Zhang, M., Wang, Y., Wang, J., Li, P., 2022. Algorithm and system co-design for efficient subgraph-based graph representation learning. arXiv preprint arXiv:2202.13538 .
Yun, S., Kim, M., Lee, J., Kang, J., Kim, H.J., 2021. Neo-gnns: Neighborhood overlap-aware graph neural networks for link prediction, in: Advances in Neural Information Processing Systems, pp. 13683–13694.
Zhang, M., Chen, Y., 2018. Link prediction based on graph neural networks, in: Advances in Neural Information Processing Systems.
Zhang, Z., Sun, S., Ma, G., Zhong, C., 2023. Line graph contrastive learning for link prediction. Pattern Recognition 140, 109537.
Zhou, T., Lü, L., Zhang, Y.C., 2009. Predicting missing links via local information. The European Physical Journal B 71, 623–630.
Zhu, J., Zhou, Y., Ioannidis, V., Qian, S., Ai, W., Song, X., Koutra, D., 2023. Spottarget: Rethinking the effect of target edges for link prediction in graph neural networks. arXiv preprint arXiv:2306.00899 .
Zhu, Z., Zhang, Z., Xhonneux, L., Tang, J., 2021. Neural bellman-ford networks: A general graph neural network framework for link prediction, in: Advances in Neural Information Processing Systems, pp. 29476–29490.

ORCID Information

Scalable Edge-Centric Subgraph Learning via Pool Walks for Link Prediction

Manizheh Ranjbar[1,*], Parham Moradi[1], Xiaodong Li[2], Mahdi Jalili[1]

1: School of Engineering, RMIT University, Melbourne, Australia

2: School of Computing Technologies, RMIT University, Melbourne, Australia

*: Corresponding author

Emails:

Manizheh Ranjbar: S3985835@student.rmit.edu.au

Parham Moradi: parham.moradi@rmit.edu.au

Xiaodong Li: xiaodong.li@rmit.edu.au

Mahdi Jalili: mahdi.jalili@rmit.edu.au

Corresponding Author:

Name: Manizheh Ranjbar
Affiliation: School of Engineering, RMIT University, Melbourne, Australia
Address: 124 La Trobe Street, Melbourne VIC 3000, Australia
Email: S3985835@student.rmit.edu.au
ORCID ID: https://orcid.org/0000-0002-9455-1959

Acknowledgements

Mahdi Jalili is supported by the Australian Research Council through projects DP240100963 and IM240100042.
The authors also acknowledge the RMIT Advanced Cloud Ecosystem (RACE) for access to GPU computing resources used in this study.

Declaration of Interest

The authors declare no competing financial or personal interests that could influence the work reported in this manuscript.

Highlights

- Introduces PWLP, a scalable influence-aware subgraph learning framework for link prediction.
- Uses pool-walk exploration to identify and prioritize high-influence nodes efficiently.
- Generates compact subgraphs that capture both local and long-range structural patterns.
- Employs lightweight line-graph encoding for expressive edge-centric representation learning.
- Achieves superior performance to state-of-the-art baselines across diverse benchmark datasets.

Declaration of interests

☒The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

☐The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: